Explaining Defeasible Entailments: A Lexicographic Closure User Interface

Samukelisiwe Zwane
zwnsam011@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Defeasible reasoning is a form of non-monotonic reasoning that allows conclusions to be revised when new or more specific information is introduced. Lexicographic Closure is an entailment algorithm that handles these exceptions while preserving as much relevant information as possible. Existing tools often focus on the final entailment result rather than clearly showing how that result was reached. This paper presents the Lexicographic Closure component of PDR-Tool, a web-based pedagogical debugger-like tool for explaining defeasible reasoning. The tool provides a step-through interface that presents the reasoning process alongside the formal algorithm and plain-English explanations. In particular, it exposes the intermediate checks performed when Lexicographic Closure resolves conflicting information. The implementation was evaluated for correctness against existing implementations and for faithfulness by comparing the displayed steps with the reasoning performed by the backend. The results show that the tool produces the expected entailment results while providing a detailed and faithful representation of the Lexicographic Closure process.

Keywords

Defeasible Reasoning, KLM Framework, Lexicographic Closure, Rational Closure, Artificial Intelligence, Pedagogical Tool, TweetyProject

1 Introduction

Humans and machines differ in the way in which they process information and derive conclusions based on their domain knowledge. Given new information, it is intuitive for humans to come up with refined conclusions; however, it is not as simple for machines. Knowledge Representation and Reasoning (KRR) is a subfield of Artificial Intelligence that uses formal logic to represent knowledge symbolically, allowing computers to draw inferences from that explicit knowledge. [2]

Classical propositional logic offers a simple framework for reasoning in KRR, known as classical reasoning, where a conclusion follows only if it is true in every case in which the stated knowledge is true. [1, 13]. Its main weakness, however, is that it cannot handle exceptions as it treats every statement the same and cannot show when one rule is a general rule and another is an exception to it. For example, while most cats are active and like Mondays, Garfield represents an exception, as he is neither active nor fond of Mondays. In classical logic this can be represented as follows:

$$\mathcal{K} = \left\{ \begin{array}{l} \text{cat} \rightarrow \text{active}, \\ \text{cat} \rightarrow \text{likesMondays}, \\ \text{garfield} \rightarrow \text{cat}, \\ \text{garfield} \rightarrow \neg\text{active}, \\ \text{garfield} \rightarrow \neg\text{likesMondays} \end{array} \right\}$$

The classical inference rules show that the knowledge base explicitly states that garfield $\rightarrow \neg$active, but also implies garfield $\rightarrow$ active because Garfield is a cat. The rules assign him conflicting traits, so the only way to keep the knowledge base consistent is to conclude that Garfield does not exist. Classical reasoning is restrictive for this reason, as adding information about an exception does not replace the more general conclusion that was already drawn. Defeasible reasoning addresses this by allowing for typicality. Instead of stating that all cats are active, for example, we can state that cats are typically active while allowing Garfield to be an exception. Unlike classical reasoning, defeasible reasoning treats conclusions as conditional and holds them by default until additional precise information is available [23].

The algorithms that compute this entailment sort the statements according to how exceptional they are into structures called ranks. The more general statements are placed in lower ranks and the more specific ones in higher ranks.

Rational Closure is the most conservative approach to computing this kind of entailment [17]. When a rank of statements conflicts with a query, the entire rank is discarded, even if only one statement causes the conflict. This loss of unrelated information is known as the drowning problem [4, 5]. Lexicographic Closure, proposed by Lehmann [16], solves this by evaluating subsets of the rank and keeping as much information as possible while removing the conflict.

Making this reasoning clear is critical because the final entailment result does not indicate how the rank was weakened or which information was rejected along the way. In a broader sense, explainable AI advocates for systems that provide the reasoning behind an outcome. For a pedagogical tool, the explanation must also be faithful to the computation, in that the steps presented to the user should be the same as those used to produce the result. [18]

Existing pedagogical tools for logic only cover classical reasoning, and the defeasible reasoners that implement Rational, Lexicographic and Relevant Closure are built for research and only show the entailment results. A user without research-level expertise therefore has no way to see the steps leading to that conclusion. This project develops a web-based pedagogical system that exposes these computational steps. The broader system supports Rational Closure [19], Lexicographic Closure, and Relevant Closure [7], while this paper focuses specifically on the Lexicographic Closure component of the step-through explanatory user interface. The aim

of this work is to determine how much of Lexicographic Closure's subset-based rank-weakening process can be exposed through a step-through interface while remaining faithful to the algorithm and preserving the correctness of the entailment result.

The rest of this paper is structured as follows. Section 2 provides the theoretical background needed to understand Lexicographic Closure. Section 3 explains the system's design. Section 4 describes how we implemented the Lexicographic Closure reasoner and how we represent its trace. Sections 5 and 6 evaluate the tool and discuss the results. Section 7 concludes the paper.

2 Background

2.1 Propositional Logic

Propositional logic is a formal language for representing and reasoning about statements. Individual statements are represented using propositional atoms, which are assigned a truth value of True T or False F [9]. The meaning of statements is defined by the truth value assigned to them through a function called a valuation. Atoms can be combined with the connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$ and $\leftrightarrow$ to form more complex formulas [1]. A knowledge base $\mathcal{K}$ is a finite set of formulas that represents information about the real world [14]. A valuation u satisfies a formula α, written $u \Vdash \alpha$, if α evaluates to true under the truth values assigned by u. Such satisfactions are the foundation for defining logical consequence and the concept of entailment extends this to a relation between a set of formulas and a formula. A formula α is entailed by a set of formulas S, denoted $S \models \alpha$, if every valuation that satisfies S also satisfies α [14]. In other words, there is no valuation that makes all formulas in S true while making α false.

While propositional logic offers a precise and compact framework for representing and evaluating logical relationships, classical entailment treats every formula in the knowledge base as exception-free [3]. It is monotonic, in that once a conclusion follows from a knowledge base, adding more information cannot cause that conclusion to be withdrawn [13, 25]. If $\mathcal{K} \models \alpha$, then $\mathcal{K} \cup \mathcal{K}' \models \alpha$ for any set of formulas $\mathcal{K}'$ [23].

While this ensures inference stability, it falls short in scenarios involving incomplete knowledge, where new information often contradicts previous conclusions. In the example from Section 1, adding garfield $\rightarrow \neg$active does not override garfield $\rightarrow$ active, which still follows from garfield $\rightarrow$ cat and cat $\rightarrow$ active. Both hold at once, leading to the conclusion $\neg$garfield. Reasoning about defaults and exceptions therefore requires a non-monotonic approach, allowing for the revision of beliefs when the default assumptions they are based on are invalidated [23].

2.2 Defeasible Reasoning

Defeasible reasoning draws conclusions that are supported by the available evidence but remain open to revision when new and possibly contradictory information arrives [22]. Unlike classical reasoning, defeasible reasoning treats conclusions as conditional, holding them by default unless they are invalidated by more specific information [23].

The KLM Framework introduced by Kraus, Lehmann and Magidor [15, 16] provides a formal approach to this type of reasoning. It extends propositional logic with a preferential consequence relation. If a and b are formulae, then $a \mathrel{|\!\sim} b$ reads "a typically implies b." Such a statement is defeasible since it explains what is typically true, while allowing more specific information to override it.

Returning to the example from Section 1, the defeasible version of the knowledge base is

$$\mathcal{K} = \left\{ \begin{array}{l} \text{garfield} \rightarrow \text{cat}, \\ \text{cat} \mathrel{|\!\sim} \text{active}, \\ \text{cat} \mathrel{|\!\sim} \text{likesMondays}, \\ \text{cat} \mathrel{|\!\sim} \text{hasFur}, \\ \text{garfield} \mathrel{|\!\sim} \neg\text{active}, \\ \text{garfield} \mathrel{|\!\sim} \neg\text{likesMondays} \end{array} \right\}$$

This knowledge base is now satisfiable as the statements cat $\mathrel{|\!\sim}$ active and garfield $\mathrel{|\!\sim} \neg$active no longer contradict one another. Garfield can therefore be treated as an exception without invalidating the general default about cats.

The KLM framework represents these preference relations using ranked interpretations to decide which entailments follow from a knowledge base. These ranked interpretations order valuations from most typical to least typical [14, 17]. We write $\mathcal{K} \mathrel{|\!\approx} \alpha \mathrel{|\!\sim} \beta$ when $\alpha \mathrel{|\!\sim} \beta$ follows from $\mathcal{K}$.

Rational Closure and Lexicographic closure use the ranking of statements to compute what follows from a knowledge base. To perform the required classical checks, a defeasible knowledge base must first be materialised. The materialisation of a defeasible knowledge base replaces every defeasible implication $\alpha \mathrel{|\!\sim} \beta$ with the classical implication $\alpha \rightarrow \beta$, written $\overrightarrow{\mathcal{K}}$. This translates the problem into a classical representation that the algorithms can use to perform exceptionality and entailment checks with a SAT solver [17].

2.3 Base Rank and Rational Closure

Rational Closure is the most conservative form of defeasible entailment [14, 15]. Anything Rational Closure entails is also entailed by the other entailment algorithms [13]. The BaseRank algorithm assigns each statement in the knowledge base a rank by iteratively identifying exceptional statements. It takes a defeasible knowledge base $\mathcal{K}$ as input and partitions it into an ordered tuple [8].

$$(R_0, \ldots, R_{n-1}, R_\infty, n).$$

At each iteration, a statement $\alpha \mathrel{|\!\sim} \beta$ is exceptional if the materialisation of the current set classically entails $\neg\alpha$, meaning that assuming α leads to a contradiction. Only the exceptional statements are carried forward to the next iteration; the non-exceptional ones are collected into R_i and assigned that rank. The process repeats until no further change occurs, at which point any remaining statements are assigned to R_∞ as classical implications [4, 8]. The full algorithm is given in Appendix A.

Applying BaseRank to the Garfield knowledge base, we substitute g for garfield, c for cat, a for active, l for likesMondays and f for hasFur. Its materialisation is

$\overrightarrow{\mathcal{K}} = \{\text{g} \rightarrow \text{c},\ \text{c} \rightarrow \text{a},\ \text{c} \rightarrow \text{l},\ \text{c} \rightarrow \text{f},\ \text{g} \rightarrow \neg\text{a},\ \text{g} \rightarrow \neg\text{l}\}$.

At the first iteration, the full materialisation entails $\neg$g, because assuming g forces c, which forces a, contradicting g $\rightarrow \neg$a. The statements with antecedent g are therefore exceptional and are

carried forward. Nothing forces ¬c, so the three cat defaults are not exceptional and are assigned to

$$R_0 = \{\mathsf{c} \mathrel{|\!\sim} \mathsf{a},\ \mathsf{c} \mathrel{|\!\sim} \mathsf{l},\ \mathsf{c} \mathrel{|\!\sim} \mathsf{f}\}.$$

The statements carried forward to the next iteration are $\mathsf{g} \rightarrow \mathsf{c}$, $\mathsf{g} \mathrel{|\!\sim} \neg\mathsf{a}$, and $\mathsf{g} \mathrel{|\!\sim} \neg\mathsf{l}$. Without the rank 0 defaults, the earlier contradiction disappears, so ¬g is no longer entailed and the two Garfield defaults are no longer exceptional. They are therefore assigned to

$$R_1 = \{\mathsf{g} \mathrel{|\!\sim} \neg\mathsf{a},\ \mathsf{g} \mathrel{|\!\sim} \neg\mathsf{l}\}.$$

At this point, only $\mathsf{g} \rightarrow \mathsf{c}$ remains. Since it is a classical statement, BaseRank assigns it to R_∞ [13]. The process therefore terminates with $n = 2$.

$$R_\infty = \{\mathsf{g} \rightarrow \mathsf{c}\}$$

Rank	Statement
R_∞	$\mathsf{g} \rightarrow \mathsf{c}$
R_1	$\mathsf{g} \mathrel{\|\!\sim} \neg\mathsf{a}$, $\mathsf{g} \mathrel{\|\!\sim} \neg\mathsf{l}$
R_0	$\mathsf{c} \mathrel{\|\!\sim} \mathsf{a}$, $\mathsf{c} \mathrel{\|\!\sim} \mathsf{l}$, $\mathsf{c} \mathrel{\|\!\sim} \mathsf{f}$

Table 1: Base ranks for the Garfield knowledge base

The cat defaults sit at rank 0 because ordinary cats are not exceptional, the Garfield defaults sit at rank 1 because Garfield is exceptional with respect to ordinary cats, and garfield → cat sits at rank ∞ as a classical statement. Although BaseRank ranks materialised statements, the ranking contains the statements in their original dematerialised form as shown above [13].

Rational Closure answers a query $\alpha \mathrel{|\!\sim} \beta$ by finding the lowest rank at which α is no longer exceptional. The RationalClosure algorithm computes this by repeatedly discarding the lowest remaining rank until α is no longer exceptional, then checking whether the remaining statements classically entail $\alpha \rightarrow \beta$ [8].

Formally, $\alpha \mathrel{|\!\sim} \beta$ is in the rational closure of $\mathcal{K}$ if

$$br_{\mathcal{K}}(\alpha) < br_{\mathcal{K}}(\alpha \wedge \neg\beta)$$

or $br_{\mathcal{K}}(\alpha) = \infty$ [11, 17], where $br_{\mathcal{K}}(\alpha)$ is the lowest rank at which α is not exceptional. The query holds when α is more typical than the situation where α holds but β does not.

For example, to evaluate $\mathcal{K} \mathrel{|\!\approx_{RC}} \mathsf{g} \mathrel{|\!\sim} \neg\mathsf{a}$, we check whether g is exceptional. It is, because the materialised statements $\mathsf{g} \rightarrow \mathsf{c}$ and $\mathsf{c} \rightarrow$ a together contradict $\mathsf{g} \rightarrow \neg\mathsf{a}$, so R_0 is discarded. Only R_1 and R_∞ remain, g is no longer exceptional, and these statements entail $\mathsf{g} \rightarrow \neg\mathsf{a}$, so the query holds.

This rank-removal strategy is excessive. When g is exceptional, the entire rank $R_0 = \{\mathsf{cat} \mathrel{|\!\sim} \mathsf{active},\ \mathsf{cat} \mathrel{|\!\sim} \mathsf{likesMondays},\ \mathsf{cat} \mathrel{|\!\sim} \mathsf{hasFur}\}$ is discarded, even though only the first two were involved in the conflict. As a result, the query $\mathsf{g} \mathrel{|\!\sim} \mathsf{f}$ (Garfield typically has fur) cannot be concluded by Rational Closure, since $\mathsf{cat} \mathrel{|\!\sim} \mathsf{hasFur}$ was discarded along with the first rank. This is the drowning problem, which motivates the lexicographic approach.

2.4 Lexicographic Closure

Lexicographic Closure, proposed by Lehmann [16], strengthens Rational Closure by preserving more information when a rank causes a conflict. Rather than immediately discarding the whole rank, it considers which parts of that rank can remain while making the query antecedent possible. This follows from the presumptive reasoning principles defined by Lehmann [14, 16].

- **Presumption of Typicality** states that typical outcomes are assumed unless there is evidence against them.
- **Presumption of Independence** states that unrelated properties do not affect each other.
- **Priority to Typicality** resolves conflicts between typicality and independence. When these lead to different conclusions, the result that is based on typical cases is preferred.
- **Respect for Specificity** states that more specific information takes priority over general information.

Lexicographic closure resolves conflicts between defaults by preferring conclusions that violate fewer and less serious rules [16]. Seriousness is decided by a comparison between subsets of the knowledge base. Two subsets are compared by how many statements they discard at each rank, starting with the more specific, higher ranks. So losing a statement from a more specific rank is considered worse than losing one from a more general rank, and subsets with more statements in more specific ranks are preferred [13, 16].

This ordering lets us define a basis for a formula. Given a defeasible knowledge base $\mathcal{K}$ and a formula α, a subset $B \subseteq \mathcal{K}$ is a basis for α if and only if the materialised counterpart $\overrightarrow{B}$ is consistent with α, that is $\overrightarrow{B} \not\models \neg\alpha$, and B is maximal with respect to the seriousness ordering [16].

A defeasible implication $\alpha \mathrel{|\!\sim} \beta$ is in the lexicographic closure of $\mathcal{K}$, written $\mathcal{K} \mathrel{|\!\approx_{LC}} \alpha \mathrel{|\!\sim} \beta$, if and only if

$$\overrightarrow{B} \cup \{\alpha\} \models \beta$$

for every basis B of α [16].

Algorithm 1 computes this entailment. It follows the same overall structure as the Rational Closure algorithm and reuses the BaseRank ranking. The key difference is in the inner loop: instead of discarding the entire rank R_i when the antecedent is refuted, Lexicographic Closure weakens the rank by replacing it with $R_{i,m}$, the disjunction of the conjunctions of every subset of R_i of size m.

The formula $R_{i,m}$ represents all ways of keeping m statements from the rank. If the antecedent is still refuted at that subset size, the value of m is reduced by one and the rank is weakened again. This continues until the largest subset size is found for which the antecedent is no longer refuted. The weakened rank is then added back to R, allowing the algorithm to preserve as much information from the original rank as possible while resolving the conflict [4, 16].

To illustrate this process, consider the Garfield knowledge base and the query $\mathsf{g} \mathrel{|\!\sim} \mathsf{f}$.

The BaseRank ranking in Table 1 gives $R_0 = \mathsf{c} \mathrel{|\!\sim} \mathsf{a}, \mathsf{c} \mathrel{|\!\sim} \mathsf{l}, \mathsf{c} \mathrel{|\!\sim} \mathsf{f}$ and $R_1 = \mathsf{g} \mathrel{|\!\sim} \neg\mathsf{a}, \mathsf{g} \mathrel{|\!\sim} \neg\mathsf{l}$. Since the full ranking refutes g, the algorithm begins by weakening R_0. With $|R_0| = 3$, the algorithm first considers subsets of size $m - 1$. All three size-2 subsets still refute g because each retains at least one of the conflicting defaults $\mathsf{c} \rightarrow$ a

Algorithm 1: LexC – Lexicographic Closure

Input: A defeasible knowledge base $\mathcal{K}$ and a defeasible query $\alpha \mid\sim \beta$

Output: true if $\mathcal{K} \mid\approx_{LC} \alpha \mid\sim \beta$, false otherwise

$(R_0, \ldots, R_{n-1}, R_\infty, n) :=$ BaseRank($\mathcal{K}$);
$R := \bigcup_{j=0}^{n} R_j$;
$i := 0$; $m := 0$;
while $\overrightarrow{R_\infty} \cup \overrightarrow{R} \models \neg\alpha$ ***and*** $R \neq \emptyset$ **do**
  $R := R \setminus R_i$;
  $m := |R_i| - 1$;
  $R_{i,m} := \bigvee_{X \in \text{Subsets}(R_i,m)} \bigwedge_{x \in X} x$;
  while $\overrightarrow{R_\infty} \cup \overrightarrow{R} \cup \{\overrightarrow{R_{i,m}}\} \models \neg\alpha$ ***and*** $m > 0$ **do**
    $m := m - 1$;
    $R_{i,m} := \bigvee_{X \in \text{Subsets}(R_i,m)} \bigwedge_{x \in X} x$;
  $R := R \cup \{R_{i,m}\}$;
  $i := i + 1$;
return $\overrightarrow{R_\infty} \cup \overrightarrow{R} \models \alpha \rightarrow \beta$;

or c → l. The value of m is therefore further reduced and at $m = 1$, the subsets containing only c → a or c → l still refute g, while c → f does not. The rank can therefore be weakened at this subset size and $R_{0,1}$ is added back to R. The final entailment check then gives

$$\mathcal{K} \mid\approx_{LC} \text{g} \mid\sim \text{f}$$

where $\mathcal{K} \mid\not\approx_{RC} \text{g} \mid\sim \text{f}$. Lexicographic Closure therefore concludes that Garfield typically has fur, while Rational Closure does not. Making this weakening process visible, step by step, is the focus of the tool presented in this paper.

3 Related work

Pedagogical tools have been developed to make formal logic easier to work through. Iltis [10], for example, teaches propositional logic through structured tasks and feedback that allow a user to work through the reasoning step by step. These tools mainly focus on classical monotonic reasoning and do not cover defeasible entailment.

SILK [12] traces defeasible reasoning paths and lets the user see the justifications behind a conclusion, but those justifications are overly symbolic, so a novice user cannot easily follow them nor the steps leading to the conclusion.

Other tools focus on computing defeasible entailment. EXTRC [6, 20] is a web application that displays the base ranking alongside the entailment result, with explanations in both mathematical and plain English terms. KLM-algorithms [13] is a web application that supports Rational, Lexicographic and Relevant Closure, with a justification-based explanation for each result that reports which statements support the conclusion and the ranks they came from. These systems provide more information about the result, but their presentation of Lexicographic Closure does not expose the weakening procedure as a discrete trace.

In EXTRC, the weakened rank is shown at different subset sizes, but the checks are presented together and do not show the order in which a larger subset size was tried and rejected before moving to the next one. KLM-algorithms exposes the sub-knowledge bases and identifies those that are consistent with the query antecedent, but does not show each failed check that caused the algorithm to continue weakening the rank. Hamayobe also notes that the system assumes a moderate level of KLM knowledge and identifies greater transparency around statement selection and removal, more granular explanations, and visual traces of the reasoning process as areas for future improvement [13].

This paper addresses these limitations by separating the weakening process into individual steps and presenting each step with its formal definition alongside a plain English explanation. For Lexicographic Closure, the tool shows every subset checked at each subset size, records whether each one still refutes the query antecedent, and shows why the algorithm either keeps that subset size or continues to a smaller one and provides a detailed step-through explanation of entailments.

4 System Design and Implementation

This section describes what the tool does, the decisions behind the Lexicographic Closure component, and how a user works through a run of the algorithm. The tool is built by adapting and extending existing implementations built on the same reasoning tools, exposing algorithmic depth and therefore contributing to the understandability of the algorithms. A more detailed breakdown is shown below.

Component	Origin
Propositional syntax and SAT solving	TweetyProject [26]
Controller, reasoner factory, query parsing, shared entailment model	Adapted from earlier honours projects [20]
BaseRank implementation and step-through	Adapted [6, 20], shared across the group
Input page, Information and Tutorial pages	Shared across the group
Lexicographic Closure pseudocode shown in the interface	Algorithm 3.5 [13]
Subset enumeration	Adapted from the 2024 implementation [20]
Lexicographic Closure reasoner with per-subset checks	This work
Trace of every subset tested and its outcome	This work
Lexicographic Closure step-through and its panels	This work

Table 2: Contributions

For Lexicographic Closure, the main contribution is the per-subset checking and recording, the reasoner trace, and the step-through interface used to display it. Correctness and faithful representation are essential components of a pedagogical tool, and each step must correspond exactly to the pseudocode algorithm shown. The step-through therefore needs to show which ranks are considered, which subsets are tested, why the algorithm moves to a smaller subset size, and how the final result is obtained and justified. To verify this, the outputs of the system are compared against several reference implementations.

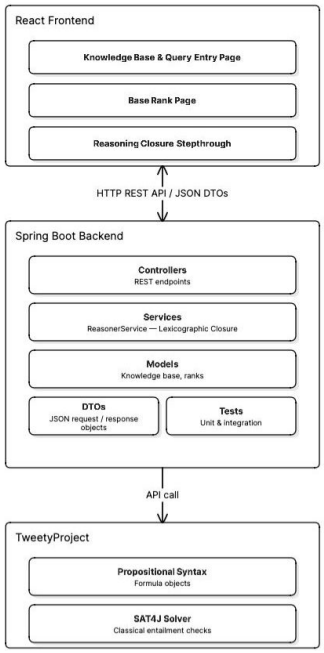


Figure 1: Architecture of the system

4.1 System Architecture

The system separates the user interface from the reasoning logic and the two components communicate over a REST API using JSON objects. The frontend is built using React and Tailwind CSS, which allows the layout, navigation controls and formula rendering to be shared across all three closures. The backend is written in Java using Spring Boot because the reasoning layer is built on TweetyProject, which is a collection of Java libraries for knowledge representation and reasoning [26]. TweetyProject supplies the propositional syntax objects used throughout the backend and a SAT solver that answers classical entailment checks. Every closure algorithm in this project reduces defeasible entailment to a sequence of these classical checks. The reasoning itself happens entirely within the backend. The results and trace are sent to the frontend through JSON objects, ensuring that the step-through remains faithful to the execution that produced the entailment result. This separation supports modularity and allows each layer to be tested individually [24]. Each reasoner implements a common ReasonerService interface, so entailment algorithms can be added without changing the frontend.

4.2 Interface design

The user interface focuses on how the user provides the input and follows the reasoning process. It is divided into two main parts: the input view and the step-through view. When entering the application, the user first builds a knowledge base and a query. The knowledge base can be entered as a comma-separated list of formulas, uploaded as a file, or selected from the preloaded examples. Formulas are validated in real time, and invalid formulas are reported below the input box. This makes errors immediately visible and gives the user a clear indication of what needs to be corrected [21]. The query is built using an antecedent, a relation, and a consequent. The relation can either be classical or defeasible, and the consequent can optionally be negated. Both the knowledge base and query entry panels include a help button that provides a more detailed explanation of what is expected as input from the user, following Nielsen's help and documentation heuristic [**?**].

Once the knowledge is loaded, an algorithm selector lets the user choose between Rational Closure, Lexicographic Closure, Basic Relevant Closure, and Minimal Relevant Closure. The system displays the entailment result and execution time for each selected algorithm. The execution times are shown because the algorithms differ in computational cost, with the performance trade-off introduced by the Lexicographic Closure design discussed further in Section 4.3. The user can then choose to step through one specific algorithm or compare the algorithms side by side.

The base ranking is computed once for a knowledge base and shared by each entailment algorithm. The comparison page can therefore show what each algorithm keeps, weakens, or removes without differences being caused by different BaseRank computations.

The step-through view goes through the algorithmic detail of how the result was reached. These pages contain a header that displays the algorithm that is currently being explained, a banner showing the BaseRank result and the user query, and three main panels. A visualisation of the ranking sits at the top and displays the current state of the ranked knowledge base. Below it, the algorithm's pseudocode is shown alongside a plain-language explanation of the current step. For Lexicographic Closure, the pseudocode follows Algorithm 3.5 as presented by Hamayobe [13]. This side-by-side placement of the algorithm and the explanation is done so that the formal and informal account of a step can always be visible, supporting both the expert and novice users. Consistent terminology and panel layouts are used across the different algorithms so that the interface remains familiar when moving between them [21]. The system also has a responsive layout, to ensure accessibility and usability across devices.

Additional pedagogical depth is provided through the information and help pages. The information page shows the purpose of the tool, defines terms that are used throughout the tool, and gives a more detailed explanation of the syntax and semantics in plain English. The user also has the option to view the more detailed mathematical definitions of those concepts. It further explains how knowledge bases and queries are structured and gives explicit instructions on the edge cases accepted by the system. The help page is a tutorial page that walks the user through the tool using an example.

4.3 Design decisions for Lexicographic Closure

The system has to show why a rank was weakened in the way that it was, and this shaped how the Lexicographic Closure reasoner handles subsets and records its decisions. When weakening a rank R_i, the algorithm constructs

$$R_{i,m} = \bigvee \{ \bigwedge X \mid X \subseteq R_i,\ |X| = m \}$$

and performs one classical entailment check on this formula. This is the combined formula approach used by the 2024 implementation [20]. It is enough to compute the entailment result, but it does not show which subsets were responsible for it. The disjunction only tells us that at least one subset of size m worked, and a tool that explains why a rank was weakened has to show which one it was and what the others were. Hamayobe describes the sub-knowledge base approach and notes that it is suited to explaining an entailment result in more detail, which is why the system uses it [13].

The reasoner therefore generates the subsets individually and checks if each corresponding sub-knowledge base, $R_\infty \cup R \cup X$, still refutes the query antecedent. By checking the subsets separately, the system can record which subsets still refute the antecedent and give the step-through interface the information needed to explain why the algorithm keeps the current subset size or moves to a smaller one. The trade-off of this explainability, however, is that the system performs one satisfiability check for every subset and not one for each subset size.

The trace is recorded while the algorithm is running so that the interface only has to show what the reasoner did. Each subset check and the decisions made while weakening a rank are stored as they occur, and the completed trace is returned with the entailment result. The interface can then simply display this information, keeping the explanation consistent with the reasoning performed by the backend.

Because the trace records each check, the subsets that fail are kept as well. If no subset of size m avoids refuting the antecedent, m is reduced by one and the subsets are checked again. Keeping these failed checks explains why the algorithm had to continue weakening the rank, expanding on the granularity of the tool's explanations.

The final decision was how much of this process should be recorded as one step in the interface. Showing each step for all the subsets would make the trace unnecessarily long, while using only the outer loop, one step per rank, would hide the changing subset size. The tool therefore uses one pass of the inner loop of Algorithm 1 as a step. Each step corresponds to one subset size and shows all the subsets of that size as well as the outcome of each check. This keeps the weakening of the rank visible, without making each individual subset its own step.

4.4 Worked example

A run of the tool on the Garfield knowledge base from Section 2.2 shows what the step-through exposes. The knowledge base and query are entered on the home page as shown in Figure 2. In this example, Rational Closure and Lexicographic Closure are selected. The initial results show that Rational Closure does not entail that Garfield typically has fur, exposing the drowning problem, while Lexicographic Closure does. The user can then select Lexicographic Closure to step through how this result was obtained. The panel also shows the execution times for each algorithm, to indicate the cost of the different algorithms.

Before the Lexicographic Closure computation begins, the user steps through the BaseRank construction. Figure 3 shows the final ranking produced for the Garfield knowledge base. The interface

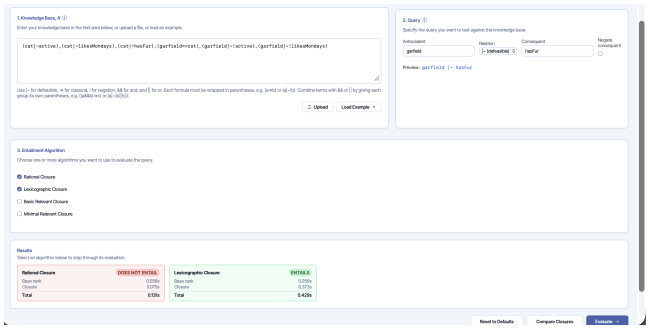


Figure 2: Knowledge base and query entry home page

presents the ranking above the BaseRank pseudocode and its explanation. During the step-through, the current lines of the algorithm are highlighted and the explanation changes according to the current step. Once the ranking is complete, the user can continue directly to the Lexicographic Closure step-through.

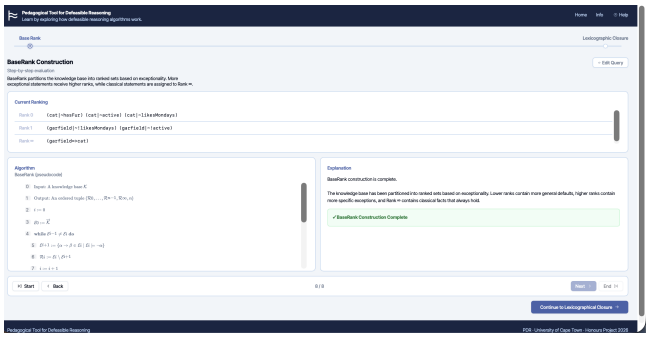


Figure 3: BaseRank construction

For this knowledge base, the Lexicographic closure algorithm is broken down into six steps. Figure 4 shows the third of these steps where all three size-two subsets still refute the antecedent garfield. The interface therefore reports that another statement must be dropped before the algorithm continues to the next subset size. The user is also given the option to reveal more detail about how $R_{0,1}$ is built as well as the full algorithm trace.

Figure 5 shows the final entailment step. The ranking panel shows that R_0 has been weakened, with its original statements struck through and the replacement formula shown below them. The combined formula $(cat \Rightarrow active) \vee (cat \Rightarrow hasFur) \vee (cat \Rightarrow likesMondays)$ is tagged as weakened, while ranks 1 and ∞ remain the same. The final check is made against the surviving sub-knowledge base and that sub-knowledge base entails the materialised query, so the result shown is that garfield $|\sim$ hasFur is entailed under Lexicographic Closure. The final panel also displays a justification for the result. The fur statement survives because it is not responsible for the conflict that makes Garfield exceptional.

5 Results

The tool is evaluated on whether it computes Lexicographic Closure entailment correctly, whether the trace shown by the interface is a



Figure 4: The first weakening attempt for R_0, m=2

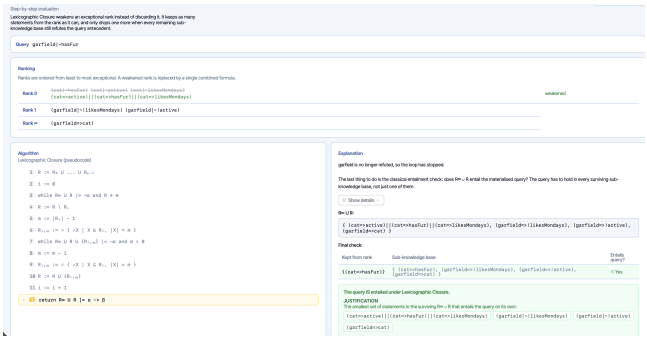


Figure 5: Final Lexicographic Closure entailment

faithful representation of the algorithm, and whether the explanations are easy enough to follow. The cost introduced by exposing the individual subset checks is also assessed. The worked example in the previous section displayed one execution of the tool, while the tests in this section consider a wider range of cases.

5.1 Correctness

Correctness was evaluated by comparing the final Lexicographic Closure entailment results with two existing implementations of Moloi [20] and Hamayobe[13]. To compare these systems, the same knowledge base and query were given to all three systems and the entailment results were compared. Expected results were also worked manually using the definition of the algorithm. The test cases include entailments, non-entailments, cases where no weakening is required, cases where a rank must be weakened, and cases where a rank is removed completely. To evaluate correctness against other implementations, the following knowledge bases and queries were used:

- $\mathcal{K}_1 = \{employee \mid\sim worksWeekdays,\ employee \mid\sim hasDesk,\ employee \mid\sim paidMonthly,\ contractor \Rightarrow employee,\ contractor \mid\sim \neg worksWeekdays,\ contractor \mid\sim \neg paidMonthly\}$, with the query $contractor \mid\sim hasDesk$. This tests a rank of three statements where two subset sizes must be checked;
- $\mathcal{K}_2 = \{vehicle \mid\sim needsFuel,\ electricCar \Rightarrow vehicle,\ electricCar \mid\sim \neg needsFuel,\ electricCar \mid\sim hasBattery,\ hybridCar \Rightarrow electricCar,\ hybridCar \mid\sim needsFuel\}$, with the query $hybridCar \mid\sim hasBattery$. This tests a knowledge base with three finite ranks;
- $\mathcal{K}_3 = \{patient \mid\sim recovers,\ criticalPatient \rightarrow patient,\ criticalPatient \mid\sim \neg recovers\}$ with the query $criticalPatient \mid\sim recovers$. This tests the removal of a rank containing one statement;
- $\mathcal{K}_4 = \{laptop \Rightarrow computer\}$, with the query $laptop \mid\sim portable$. This tests a knowledge base with no finite ranks where the query is not supported by the strict knowledge base;
- $\mathcal{K}_5 = \{robot \Rightarrow machine,\ robot \Rightarrow \neg machine\}$, with the query $robot \mid\sim machine$. This tests an antecedent that is inconsistent with R_∞.

The full test outputs are provided in Table 4. All the test cases produced the same output across the different tools, adding to the correctness of this implementation. The implementation was also tested using unit tests. These tests cover entailed and non-entailed queries, exceptional and non-exceptional antecedents, unsupported conclusions, the drowning problem, and cases where specific exceptions must override more general defaults. A summary of the entailment tests and their expected results is provided in Table 5 and under the Test folder in the code. Together these tests ensured that the different components as well as the final entailments were produced as expected.

5.2 Faithfulness of the trace

Since the purpose of the tool is to expose the reasoning process, the information displayed at each step was checked against the state recorded by the reasoner. The highlighted pseudocode was checked against the operation described in the explanation panel. To make sure the interface did not leave out a subset that the reasoner had tested, the number of sub-knowledge bases displayed at subset size m was compared to $\binom{|R_i|}{m}$. The working knowledge base shown in the explanation was also compared to the working set kept in the trace, and the ranking visualisation was checked to ensure that weakened and removed ranks reflected the state reached at that point in the computation. These checks are shown in the worked example.

5.3 Usability feedback

The tool was reviewed by members of the Logic-Based AI research group at the University of Cape Town, to check whether the reasoning shown by the interface was understandable and whether the explanations accurately described the algorithm. The feedback on our approach was positive as the algorithm was considered clear, and displaying different sub-knowledge bases during Lexicographic Closure was identified as a useful component of the interface. The

evaluation was informal as conducting formal evaluations would require ethics clearance, which was not in the scope of this project.

5.4 Performance Evaluation

The decision to expose the individual subset checks increases the number of satisfiability checks performed by the system. Performance was therefore evaluated to determine the effect of this design decision. Weakening a rank with n statements may require checking up to 2^n subsets, since several subset sizes may need to be tested and each subset requires a satisfiability check. In comparison, the combined formula approach requires at most n checks for the same rank. The cost is therefore exponential and depends on how many statements are in the rank being weakened. (I do not have any graphics for this)

6 Discussion

The results show that grouping the subsets by size gives enough detail to follow the weakening process without making the trace unnecessarily long. This removes the need for a separate interface step for each subset while still showing why the algorithm moves to a smaller subset size. The failed subsets are also a part of the explanation. They show why a subset size was rejected before the algorithm moved on. In the Garfield example, the six sub-knowledge bases considered across $m = 2$ and $m = 1$ show which combinations still conflict with $garfield$ and which one keeps the unrelated $hasFur$ statement. The correctness results show that the extra trace information does not change the final entailment result for the cases tested. The PDR tool produced the same outputs as Moloi's implementation [20] and Hamayobe's implementation [13], and the unit tests covered a wider range of entailment and weakening cases. The faithfulness checks showed that the subsets, working knowledge base, ranking state and highlighted pseudocode shown in the interface matched the state recorded by the reasoner, which is also visible in Figures 4 and 5. This was supported by positive informal feedback from the Logic-Based AI research group, particularly on the step-by-step structure and display of the sub-knowledge bases.

6.1 Limitations

No formal user testing was done, so the pedagogical explainability and understandability of the tool could not formally be measured. The feedback that was received could only determine whether the tool was usable and whether the explanations were technically clear. The use of the sub-knowledge-base approach introduces a computational limitation because each subset is checked separately. As the size of a rank increases, the number of subsets grows exponentially, which increases both the execution time and the amount of trace information shown on the interface. This makes the current tool more suitable for small knowledge bases.

6.2 Future Work

Future work can extend this system by allowing for a broader range of defeasible entailment algorithms, making it possible to compare more reasoning methods. The system could also be optimised by investigating ways to filter or reduce the number of subsets that need to be checked for larger knowledge bases, while still preserving the trace used by the interface. An AI chatbot could also be added to answer questions about the reasoning process for alternative queries during the stepping through of one query, with its responses grounded in the existing algorithms and traces so that its explanations remain faithful to the implemented methods. Other extensions include natural-language knowledge-base entry, building on Julia's work [6], and automatic detection of the drowning problem.

7 Conclusions

This work presented a step-through interface for exposing the rank-weakening process of Lexicographic Closure. The approach records every subset checked by the reasoner, including those that fail, but groups the checks by subset size when presenting them to the user. This provides enough detail to show which subsets were considered, why a particular subset size was rejected, and why the algorithm either weakens the rank further or continues to the final entailment check.

The results show that this level of granularity can be exposed while preserving the behaviour of the underlying algorithm. The tested entailment results agreed with the existing implementations of Moloi and Hamayobe, while the trace checks showed that the subsets, working knowledge base, ranking changes, and pseudocode displayed by the interface corresponded to the state recorded by the reasoner. The additional detail comes with a computational cost, since the number of subsets grows exponentially with the size of the rank being weakened. Furthermore, without a formal learner study, we cannot claim whether the interface improves learning. The contribution of this work is therefore a faithful and more granular representation of Lexicographic Closure's reasoning process that can form the basis for future evaluation and further development of pedagogical tools for defeasible reasoning.

7.1 AI Usage Declaration

AI tools, specifically ChatGPT and Claude, were used to assist with editing written sections and provided refinement and feedback during the writing process. All written content and ideas were produced by the author and edited with the assistance of AI tools. AI was also used extensively to assist with LaTeX formatting and document layout. The final decisions regarding the content, implementation, design, and presentation of the work remained the responsibility of the author.

References

[1] Mordechai Ben-Ari. 2012. *Mathematical Logic for Computer Science.* Springer.

[2] Ronald J. Brachman and Hector J. Levesque. 2004. *Knowledge Representation and Reasoning.* Morgan Kaufmann.

[3] Giovanni Casini, Thomas Meyer, Kody Moodley, and Ivan Varzinczak. 2013. Towards practical defeasible reasoning for description logics. In *Proceedings of the 26th International Workshop on Description Logics (DL 2013) (CEUR Workshop Proceedings).*

[4] Giovanni Casini, Thomas Meyer, and Ivan Varzinczak. 2019. Taking defeasible entailment beyond rational closure. In *Logics in Artificial Intelligence (JELIA 2019).* Springer, 182–197.

[5] Giovanni Casini and Umberto Straccia. 2012. Lexicographic closure for defeasible description logics. In *Proceedings of the 8th Australasian Ontology Workshop (AOW 2012).*

[6] Julia Cotterrell. 2025. *A User Interface to Aid in the Understanding of Rational Closure Entailments.* BSc (Honours) Project. University of Cape Town.

[7] Liam De Saldanha. 2026. *Explaining Defeasible Entailments: A Relevant Closure User Interface.* BSc (Honours) Project. University of Cape Town.

[8] Michael Freund. 1998. Preferential reasoning in the perspective of Poole default logic. *Artificial Intelligence* 98, 1–2 (1998), 209–235.

[9] Jean H. Gallier. 2015. *Logic for Computer Science: Foundations of Automatic Theorem Proving.* Dover Publications.

[10] Gaetano Geck, Artur Ljulin, Jonas Haldimann, Jan May, Jonas Schmidt, Marko Schmellenkamp, Dennis Sonnabend, Fabian Tschirbs, Felix Vehlken, and Thomas Zeume. 2019. Teaching logic with Iltis: An interactive, web-based system. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE 2019).* 307.

[11] Laura Giordano, Valentina Gliozzi, Nicola Olivetti, and Gian Luca Pozzato. 2015. Semantic characterization of rational closure: From propositional logic to description logics. *Artificial Intelligence* 226 (2015), 1–33.

[12] B. Grosof, M. Burstein, M. Dean, C. Andersen, B. Benyo, W. Ferguson, D. Inclezan, and R. Shapiro. 2010. A SILK graphical UI for defeasible reasoning, with a biology causal process example. In *Proceedings of the ISWC 2010 Posters and Demonstrations Track.*

[13] Chipo Hamayobe. 2026. *Implementation and Evaluation of KLM-Style Defeasible Entailment and Explanation Algorithms.* Master's thesis. University of Cape Town.

[14] Adam Kaliski. 2020. *An Overview of KLM-Style Defeasible Entailment.* Master's thesis. University of Cape Town.

[15] Sarit Kraus, Daniel Lehmann, and Menachem Magidor. 1990. Nonmonotonic reasoning, preferential models and cumulative logics. *Artificial Intelligence* 44, 1–2 (1990), 167–207.

[16] Daniel Lehmann. 1995. Another perspective on default reasoning. *Annals of Mathematics and Artificial Intelligence* 15, 1 (1995), 61–82.

[17] Daniel Lehmann and Menachem Magidor. 1992. What does a conditional knowledge base entail? *Artificial Intelligence* 55, 1 (1992), 1–60.

[18] Joao Marques-Silva and Alexey Ignatiev. 2022. Delivering trustworthy AI through formal XAI. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2022)*, Vol. 36. 12342–12350. doi:10.1609/aaai.v36i11.21499

[19] Nikita Martin. 2026. *Pedagogical Defeasible Reasoning Tool: A Rational Closure User Interface.* BSc (Honours) Project. University of Cape Town.

[20] Thabo Vincent Moloi. 2024. *Extending Defeasible Reasoning Beyond Rational Closure.* BSc (Honours) Project. University of Cape Town.

[21] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 1990).* Association for Computing Machinery, 249–256. doi:10.1145/97243.97281

[22] John L. Pollock. 1987. Defeasible reasoning. *Cognitive Science* 11, 4 (1987), 481–518.

[23] Raymond Reiter. 1980. A logic for default reasoning. *Artificial Intelligence* 13, 1–2 (1980), 81–132.

[24] Ian Sommerville. 2016. *Software Engineering* (10th ed.). Pearson Education.

[25] Alfred Tarski. 1983. *Logic, Semantics, Metamathematics: Papers from 1923 to 1938* (2nd ed.). Hackett Publishing Company.

[26] Matthias Thimm. 2017. The Tweety library collection for logical aspects of artificial intelligence and knowledge representation. *KI – Künstliche Intelligenz* 31, 1 (2017), 93–97.

A Algorithm Listings

Algorithm 2: BaseRank

Input: A knowledge base $\mathcal{K}$
Output: An ordered tuple $(R_0, \ldots, R_{n-1}, R_\infty, n)$

$i := 0$;
$E_0 := \overrightarrow{\mathcal{K}}$;
repeat
  $E_{i+1} := \{\alpha \rightarrow \beta \in E_i \mid E_i \models \neg\alpha\}$;
  $R_i := E_i \setminus E_{i+1}$;
  $i := i + 1$;
until $E_{i-1} = E_i$;
$R_\infty := E_{i-1}$;
if $E_{i-1} = \emptyset$ **then**
  $n := i - 1$;
else
  $n := i$;
return $(R_0, \ldots, R_{n-1}, R_\infty, n)$;

Algorithm 3: RationalClosure

Input: A knowledge base $\mathcal{K}$ and a query $\alpha \mid\sim \beta$
Output: true if $\mathcal{K} \mid\approx_{RC} \alpha \mid\sim \beta$, false otherwise

$(R_0, \ldots, R_{n-1}, R_\infty, n) :=$ BaseRank($\mathcal{K}$);
$i := 0$;
$R := \bigcup_{j=0}^{n-1} R_j$;
while $R_\infty \cup R \models \neg\alpha$ ***and*** $R \neq \emptyset$ **do**
  $R := R \setminus R_i$;
  $i := i + 1$;
return $R_\infty \cup R \models \alpha \rightarrow \beta$;

B System Implementation

The system uses a Java Spring Boot backend and a React frontend that communicate through a REST API using JSON objects. The backend performs the reasoning and records the execution trace, while the frontend presents these recorded steps to the user. `ReasonerController` receives an entailment request and uses `ReasonerFactory` to select the appropriate reasoner. Each reasoner implements the common `ReasonerService` interface, allowing the different closure algorithms to use the same request structure while keeping their reasoning logic separate.

B.1 Lexicographic Closure Reasoner

Lexicographic Closure is implemented in `LexicographicReasonerImpl` and follows Algorithm 1. The reasoner receives the BaseRank result and constructs the working set R from the finite ranks while keeping R_∞ separate. When a rank R_i must be weakened, `subsetsOfSize` generates its subsets of size m using the bitmask approach adapted from Moloi [20]. Unlike the earlier implementation, these subsets are kept individually so that the outcome of every check can be recorded for the interface.

For each subset size, `buildCombinedFormula` generates the sub-knowledge bases $R_\infty \cup R \cup X$ and records whether each one still refutes the query antecedent as a `SubKnowledgeBaseCheck`. At the same time, the subsets are combined to form $R_{i,m}$, which is used by the reasoner to decide whether the rank must be weakened further. Producing both in the same method means that the combined formula is used for the formal algorithm while the individual checks provide the information needed to explain that decision. The interface therefore displays the checks that were actually performed rather than reconstructing them afterwards.

B.2 Trace Representation

The trace is recorded while the reasoner executes. A `SubKnowledgeBaseCheck` records one subset test, while a `LexicographicStep` groups the checks produced while processing a rank. These are returned through `LexicographicEntailment`, together with the weakened and removed ranks and the final entailment information.

On the frontend, the checks are grouped by subset size so that one pass of the inner loop becomes one interface step. Each step therefore shows all subsets checked at that size and their outcomes, rather than creating a separate step for every subset. This keeps the reason for moving to a smaller subset size visible without making

the step-through unnecessarily long. The frontend performs no additional reasoning; it uses the trace returned by the backend to update the ranking, pseudocode, and plain-English explanation.

B.3 Trace representation and step-through interface

The reasoning and the explanation are done separately. The two classes that hold the traces are `SubKnowledgeBaseCheck`, which records a single subset test and `LexicographicStep`, which records the processing of one rank. A rank produces one `LexicographicStep` and many `SubKnowledgeBaseCheck` objects, one for every subset tried at every size attempted. Keeping them separate means a rejected subset stays visible in the trace, so the interface can show the subset sizes that failed and those that worked.

The reasoner also builds a second trace of `EntailmentStep` objects. Both traces are returned on the `LexicographicEntailment` object. The frontend works with formula strings and rank numbers, and does not depend on the TweetyProject library types.

One step is rendered by three components.

Implementation details

`ReasonerController` exposes one endpoint, where the path variable names the reasoning algorithm and the request body has the query. The controller parses the query into a `PlFormula` object, obtains the base ranking, and asks `ReasonerFactory` for the reasoner matching the requested type. The reasoner returns an `Entailment` object, which is converted to JSON for the frontend.

The `Entailment` model holds the parts of a result that every algorithm produces, namely the knowledge base, the query, the base ranking, whether the query is entailed, and a step trace. `LexicographicEntailment` extends it with the ranks that were removed, the ranks that were weakened, the final sub-knowledge base checks, and the detailed trace.

C Additional Interface Screenshots

D Algorithm correspondence

Table 3 maps the formal Lexicographic Closure algorithm to the state recorded by the implementation and the information presented by the interface.

E Correctness Unit Tests

To evaluate the correctness against other implementations, the following knowledge bases were used:

F Lexicographic Closure Unit Tests

The Lexicographic Closure reasoner was tested using JUnit tests covering final entailment results as well as the internal weakening and trace behaviour. Two main knowledge bases were used:

$$\mathcal{K}_P = \left\{ \begin{array}{l} \text{bird} \mathrel{|\!\sim} \text{flies}, \\ \text{bird} \mathrel{|\!\sim} \text{wings}, \\ \text{penguin} \Rightarrow \text{bird}, \\ \text{penguin} \mathrel{|\!\sim} \neg\text{flies} \end{array} \right\}$$

and

$$\mathcal{K}_G = \left\{ \begin{array}{l} \text{cat} \mathrel{|\!\sim} \text{active}, \\ \text{cat} \mathrel{|\!\sim} \text{likesMondays}, \\ \text{cat} \mathrel{|\!\sim} \text{hasFur}, \\ \text{garfield} \Rightarrow \text{cat}, \\ \text{garfield} \mathrel{|\!\sim} \neg\text{active}, \\ \text{garfield} \mathrel{|\!\sim} \neg\text{likesMondays} \end{array} \right\}$$

Table 5 summarises the tests of the final entailment result.

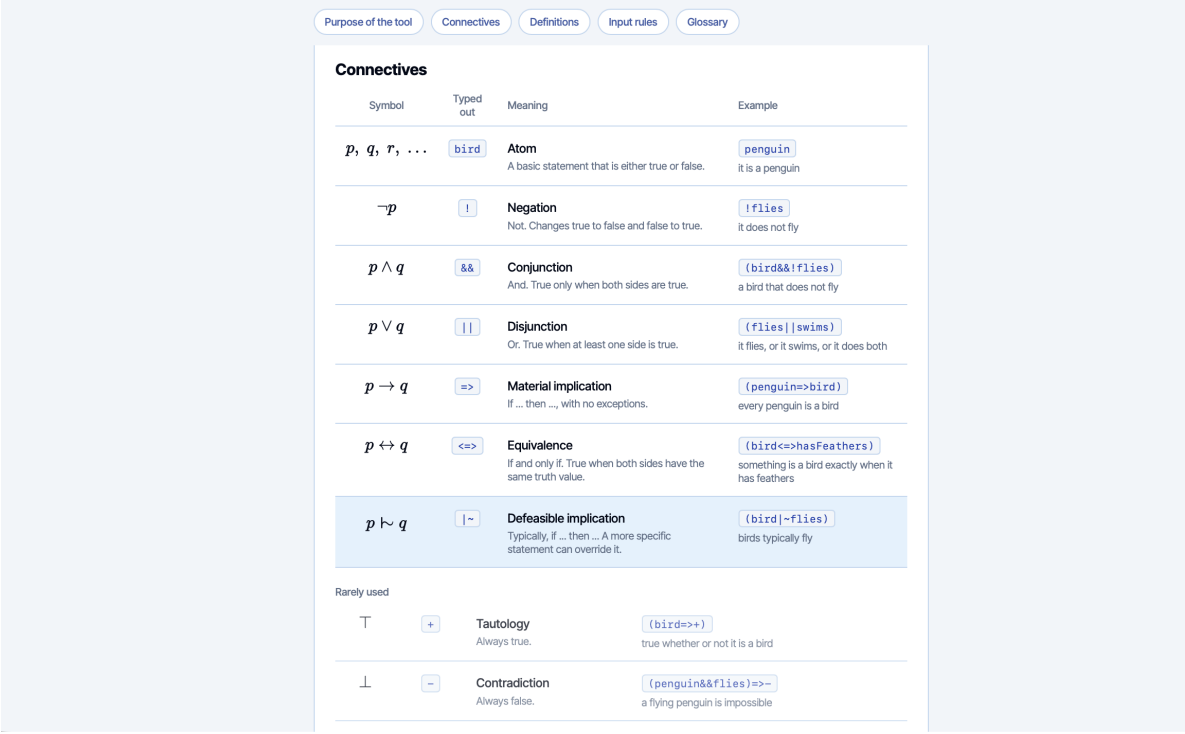


Figure 6: Connectives section of the information page.

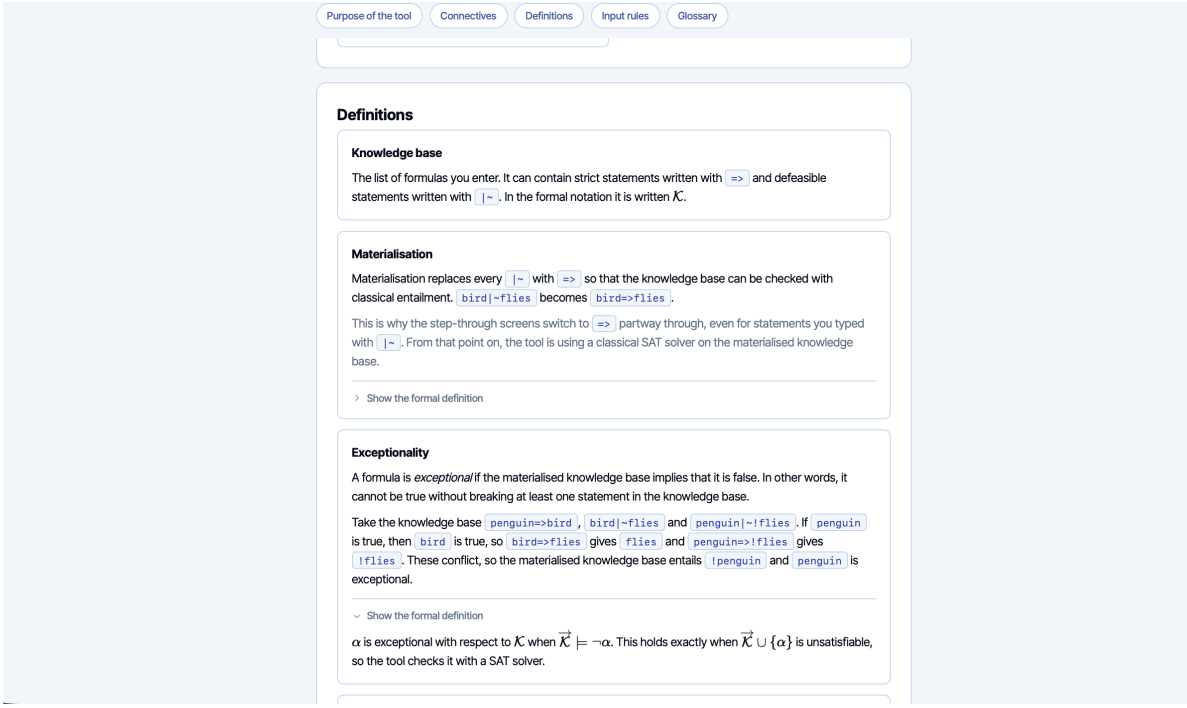


Figure 7: Definitions section of the information page.

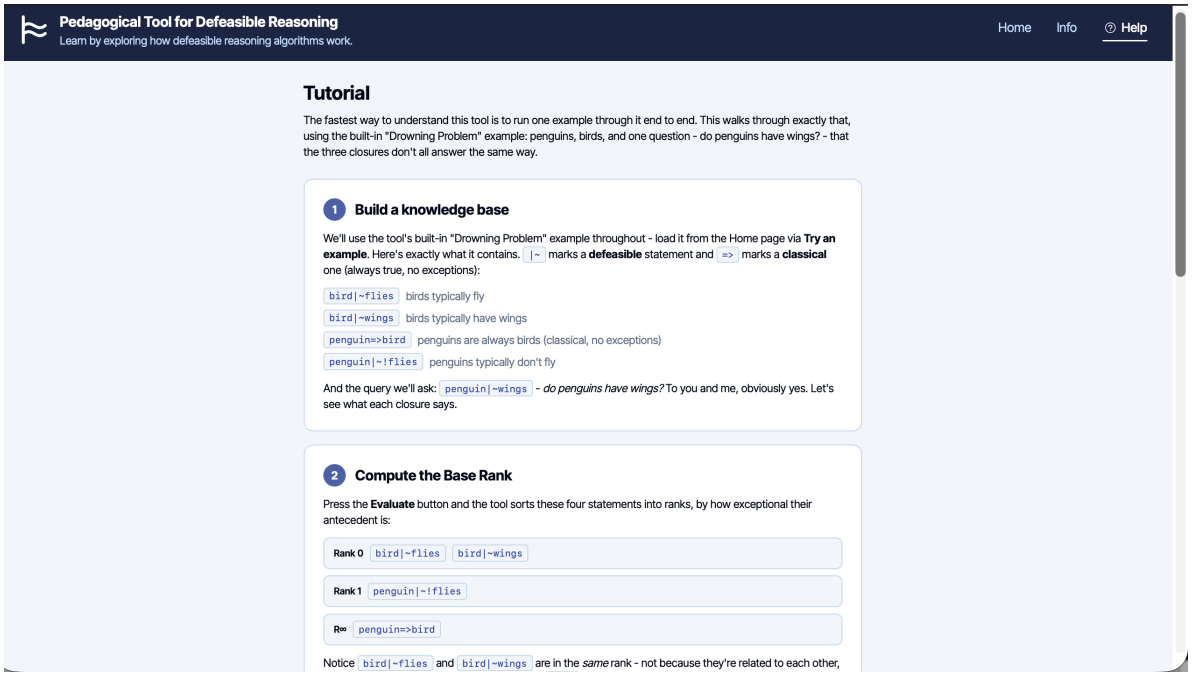


Figure 8: Tutorial page showing the worked example used to introduce the system.

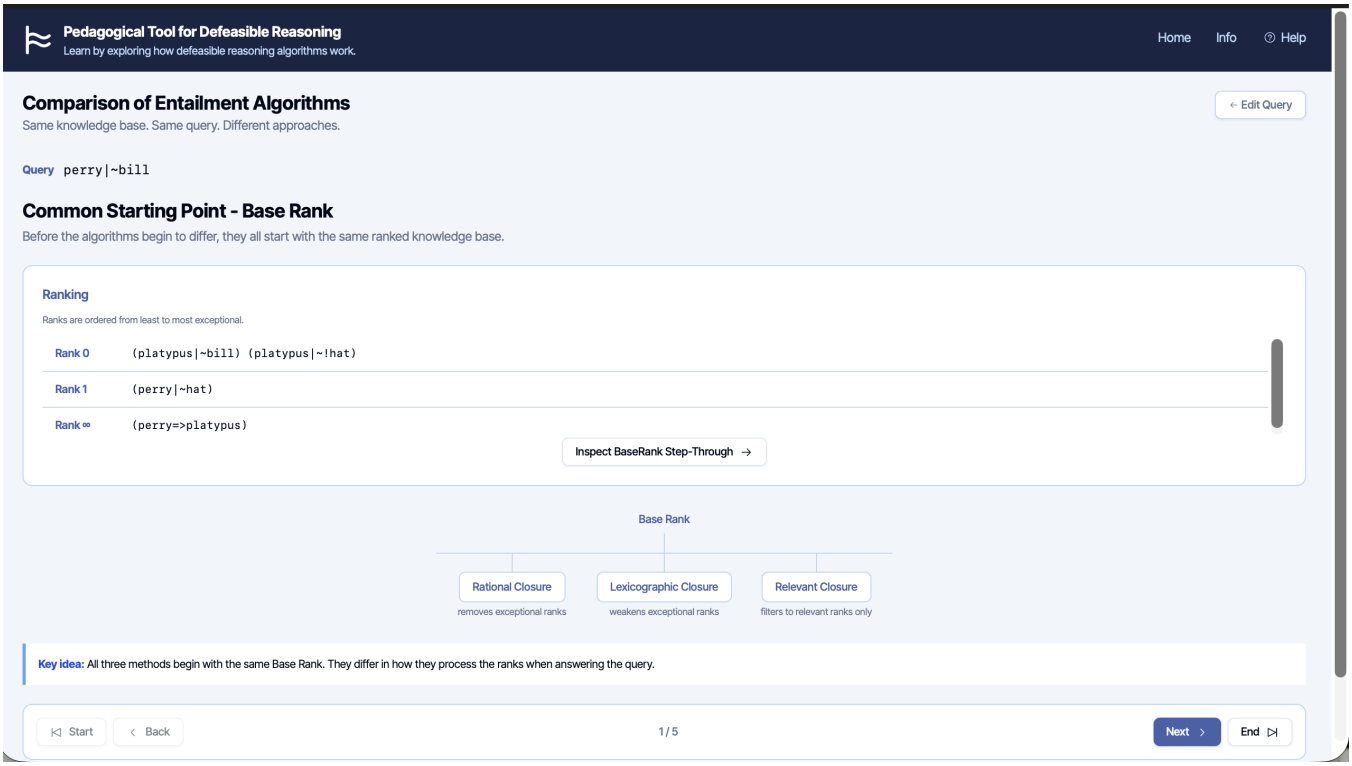


Figure 9: Comparison page showing the BaseRank ranking used by the three entailment algorithms.

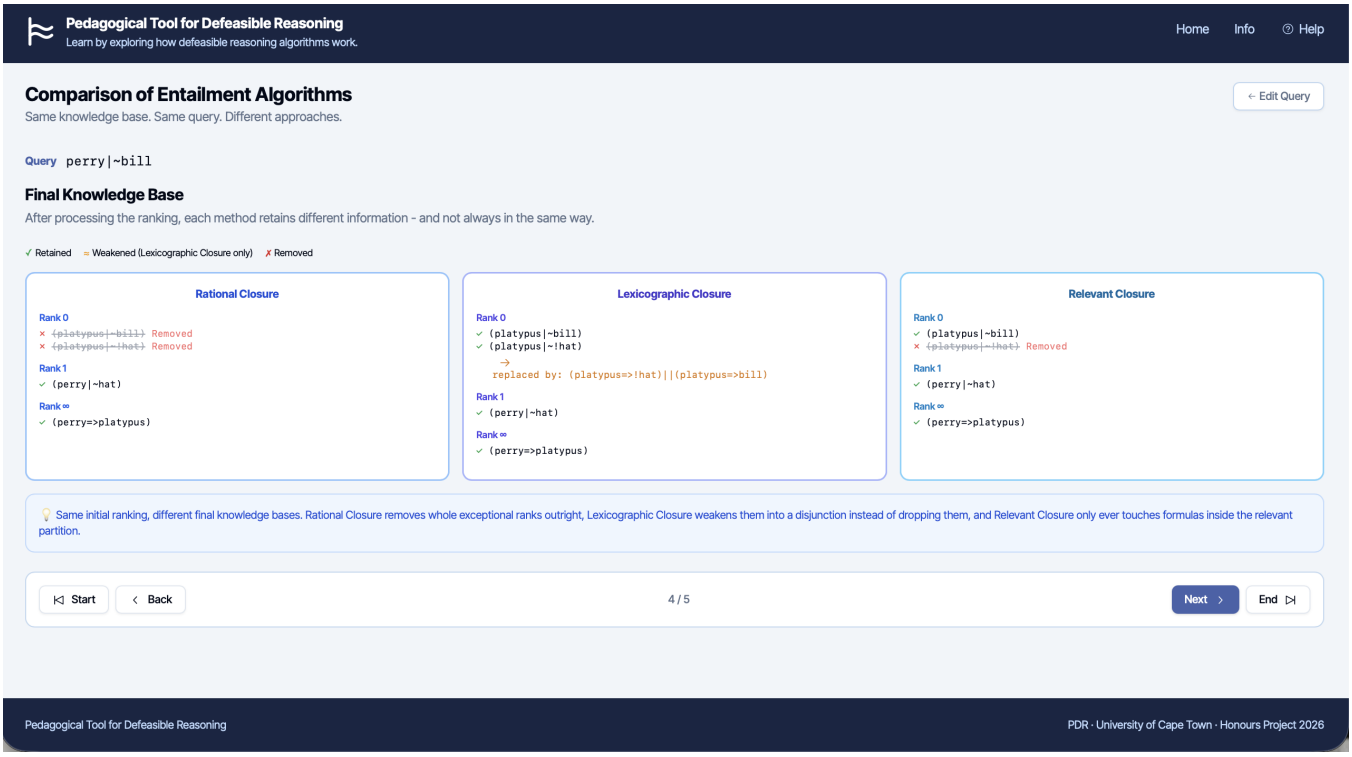


Figure 10: Comparison of the final knowledge bases produced by Rational Closure, Lexicographic Closure, and Relevant Closure.

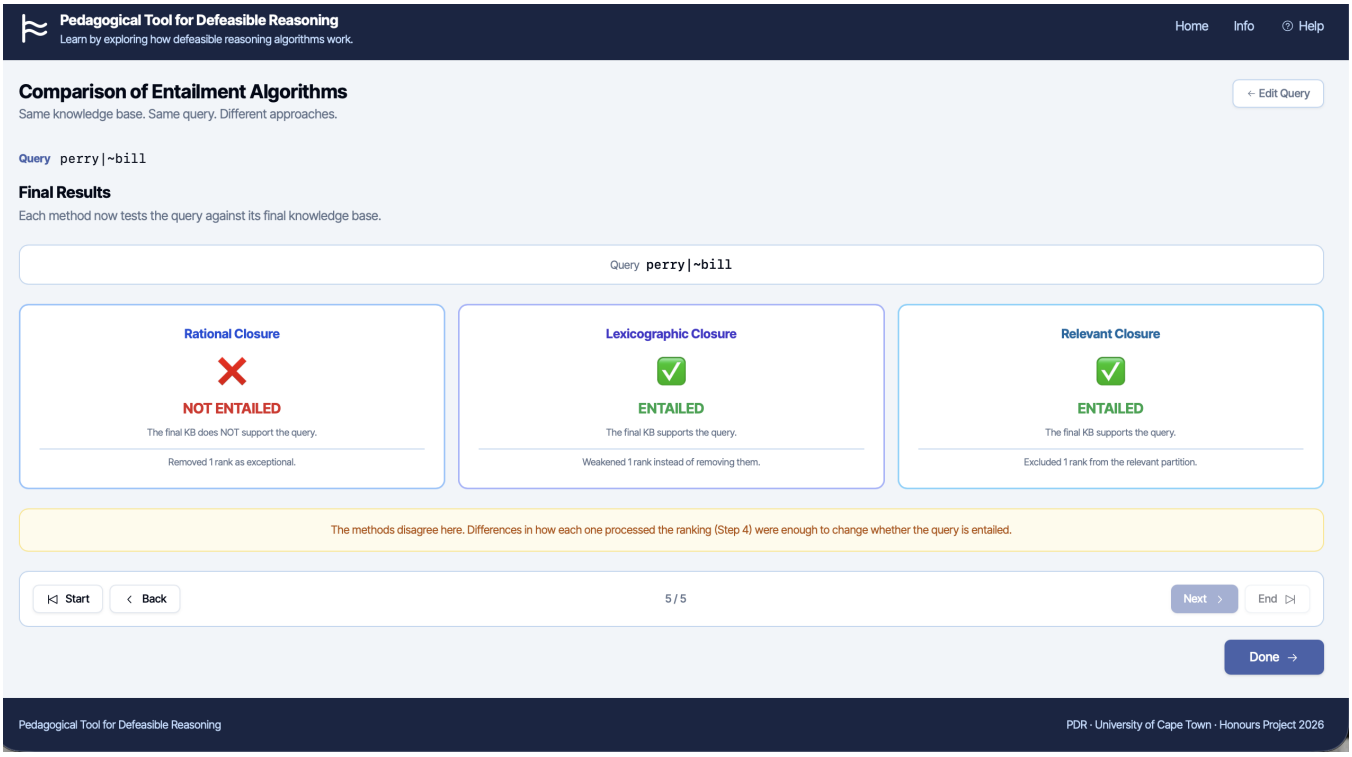


Figure 11: Comparison of the final entailment results returned by the three closure algorithms.

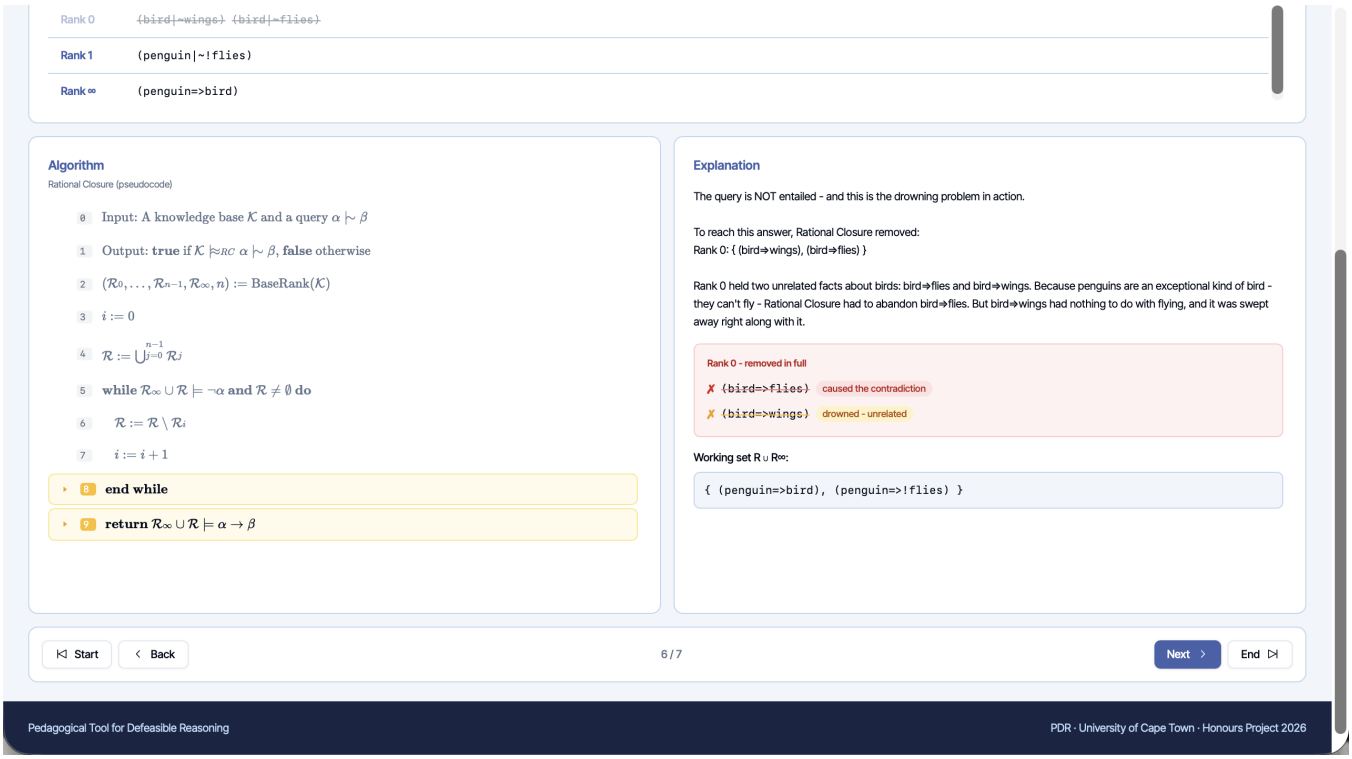


Figure 12: Rational Closure step-through illustrating the drowning problem and the removal of Rank 0.

Line	State or operation	Recorded by the implementation	What the interface shows
1	Compute BaseRank($\mathcal{K}$)	The BaseRank result stored on the common `Entailment` result	The BaseRank ranking shown before the Lexicographic Closure trace begins.
2	Initialise R with the finite ranks	The initial working ranking is built by `LexicographicReasonerImpl`	No separate step. The initial ranking already shows this state.
3	Set $i := 0$	Local loop state only	Not rendered. Initialising the rank index does not change the knowledge shown to the user.
4	Set $m := 0$	Local loop state only	Not rendered. The value is replaced with $\lvert R_i \rvert - 1$ before subset checking begins.
5	Test whether $R_\infty \cup R \models \neg\alpha$	The reasoner performs the outer-loop exceptionality check, and the result determines whether a rank must be processed	The explanation states that the antecedent is still refuted and that the current rank must be considered.
6	Remove R_i from R	The current rank is selected by `LexicographicReasonerImpl` and its processing is held in the current `LexicographicStep`	The current rank is named as the rank being weakened and the ranking display shows its removal from the working set.
7	Set $m := \lvert R_i \rvert - 1$	The first group of subset checks is generated at this size	The first weakening step shows the current subset size m.
8	Construct $R_{i,m}$	`buildCombinedFormula` builds the combined formula and stores one `SubKnowledgeBaseCheck` for every subset of size m	A table shows every subset tested at that size and whether it still refutes the antecedent. The weakened formula is also shown.
9	Test whether the weakened rank still refutes α	The combined formula is checked by the classical reasoner. The individual subset outcomes at the same size are already in the trace	The explanation shows whether the current subset size succeeds or whether the rank must be weakened further.
10	Reduce $m := m - 1$	Local loop state. The next group of subset checks is generated using the new value	The next step moves to the smaller subset size and explains why the previous size was rejected.
11	Reconstruct $R_{i,m}$	`buildCombinedFormula` generates a new group of `SubKnowledgeBaseCheck` objects for the smaller size	The next subset table shows every check at the new value of m.
12	End inner loop	Control flow only	No separate step. The outcome is already visible in the last subset-size step.
13	Add the weakened rank back to R	The weakening is recorded in `weakenedRanking`. If weakening reaches $m = 0$, the rank is recorded in `removedRanking` instead	The ranking panel shows the weakened rank, or shows that the rank has been removed completely.
14	Increment i	Local loop state only	Not rendered separately. If another rank must be processed, it begins the next trace step.
15	End outer loop	Control flow only	No separate step. The interface moves to the final entailment stage once the antecedent is no longer refuted or no finite ranks remain.
16	Check $R_\infty \cup R \models \alpha \rightarrow \beta$	The final verdict is stored on the entailment result, and `buildFinalChecks` records the sub-knowledge-base checks used for the final explanation	The final step shows the query result and the recorded checks for the final surviving state.

Table 3: Mapping between Algorithm 1, the recorded implementation state, and the Lexicographic Closure step-through interface.

KB	Query	Case tested	PDR	Moloi	Hamayobe
$\mathcal{K}_1$	$contractor \mid\sim hasDesk$	Two subset sizes	Entailed	Entailed	Entailed
$\mathcal{K}_2$	$hybridCar \mid\sim hasBattery$	Three finite ranks	Entailed	Entailed	Entailed
$\mathcal{K}_3$	$criticalPatient \mid\sim recovers$	Single-statement rank	Not entailed	Not entailed	Not entailed
$\mathcal{K}_4$	$laptop \mid\sim portable$	No finite ranks	Not entailed	Not entailed	Not entailed
$\mathcal{K}_5$	$robot \mid\sim machine$	Inconsistent antecedent in R_∞	Entailed	Entailed	Entailed

Table 4: Comparison of Lexicographic Closure entailment results across the different implementations.

Knowledge Base	Query	Expected	Behaviour tested
$\mathcal{K}_{penguin}$	$bird \mid\sim flies$	Entailed	Non-exceptional antecedent; no weakening is required.
$\mathcal{K}_{penguin}$	$penguin \mid\sim \neg flies$	Entailed	The specific exceptional statement is preserved.
$\mathcal{K}_{penguin}$	$penguin \mid\sim flies$	Not entailed	The opposite of the exceptional conclusion is rejected.
$\mathcal{K}_{penguin}$	$penguin \mid\sim swims$	Not entailed	An unsupported proposition is not entailed.
$\mathcal{K}_{penguin}$	$penguin \mid\sim wings$	Entailed	Lexicographic Closure recovers a statement lost through drowning by Rational Closure.
$\mathcal{K}_{Garfield}$	$garfield \mid\sim hasFur$	Entailed	The unrelated fur statement survives weakening of R_0.
$\mathcal{K}_{Garfield}$	$garfield \mid\sim \neg active$	Entailed	A specific Garfield exception is preserved.
$\mathcal{K}_{Garfield}$	$garfield \mid\sim \neg likesMondays$	Entailed	A second specific Garfield exception is preserved.
$\mathcal{K}_{Garfield}$	$garfield \mid\sim likesMondays$	Not entailed	The conflicting general conclusion is rejected.
$\mathcal{K}_{Garfield}$	$cat \mid\sim active$	Entailed	The Garfield exception does not change the default for ordinary cats.
$\mathcal{K}_{Garfield}$	$cat \mid\sim hasFur$	Entailed	The general cat default remains entailed.

Table 5: Unit tests of Lexicographic Closure entailment